Bounded Tool Results with Explicit Coverage: A Reproducible Transport-Conformance Study

Sami Halawa Ribas
Independent Researcher, Madrid, Spain
ORCID: https://orcid.org/0009-0002-2021-1983
Contact: sami@samihalawa.com
1 November 2026 | Version 1.0 | Technical report

Abstract

Tool-using systems often constrain a response before an agent consumes it. A syntactically valid response can nevertheless conceal omitted records, while character slicing can both invalidate JSON and exceed an intended UTF-8 byte limit. This report presents an application-level coverage envelope and an executable, deterministic comparison of four delivery policies. A factorial suite varies record count, record width, character encoding, budget and top-level field order across 512 configurations, yielding 2,048 policy observations. Byte-prefix slicing produces valid JSON in 120 configurations; character-prefix slicing does so in 160, but respects the byte budget in only 208. A record-aware prefix preserves syntax and a fixed status field in every configuration, yet provides no declaration of omitted coverage. The proposed envelope preserves those properties, explicitly reports coverage and permits exact reconstruction in all 512 configurations under exhaustive scripted retrieval. Its cost is material: a mean of 25.60 pages and 16.50 external resource reads per configuration, with up to 128 pages. These are transport-conformance results, not evidence of improved language-model accuracy, autonomous pagination or production latency. The contribution is a compact reference implementation, an inspectable synthetic corpus generator and an evaluation method that separates syntactic integrity, coverage awareness and retrieval completeness.

Keywords: agent harnesses; MCP; tool results; pagination; UTF-8; reproducibility; coverage; context engineering.

1. Problem and scope

An agent's input is mediated by serialization, budgets and orchestration. The engineering question considered here is narrow: when a structured result does not fit a delivery budget, which properties should the delivery layer preserve, and what does their preservation cost?

Recent evidence motivates checking this boundary. Sehgal, Panahandeh and Ezzati-Jivan [1] report destructive tool-output truncation in a kernel-trace diagnosis harness and show that recovering lost information can require additional work. Their study does not establish a universal loss in final-answer accuracy. This report independently implements controlled synthetic inputs; it does not reuse their traces, claim replication of their results or evaluate their harness.

MCP provides structured tool results and optional output schemas [2]. Its standard cursor-based pagination covers discovery/list operations such as tools/list and resources/list [3]. That is distinct from paging the business records returned inside a tools/call result. A custom business-result cursor therefore needs an explicit application contract; merely adding nextCursor to arbitrary output does not make it protocol-defined pagination.

We distinguish four questions: can a consumer parse the response; does it meet the delivery budget; can it tell how much remains; and can it retrieve the exact source records? None alone establishes that a model will solve a downstream task.

2. Contribution and research questions

This work contributes (i) a byte-budgeted coverage envelope, (ii) a deterministic factorial transport study with raw results and (iii) failure checks for cursor and resource handling. It does not claim that pagination, content-addressed storage or coverage metadata is new. The useful engineering contribution is combining them in a small executable contract and making their tradeoffs measurable.

RQ1 asks how prefix policies behave under UTF-8 variation and field order. RQ2 asks whether syntax-preserving record selection is sufficient to communicate completeness. RQ3 asks whether the reference envelope permits exact recovery under an exhaustive client, and at what retrieval cost. The comparison is descriptive and finite; no population-level statistical inference is made.

3. Application contract and implementation

The reference producer accepts an immutable source object containing records and a fixed proof object. Here proof is just a synthetic fixture field with state=verified and receipt=R-2026-001. Its name is not evidence of a real transaction or cryptographic attestation. The fixture makes preservation of a non-record field independently testable.

Each page contains snapshot, proof, total, start, returned, remaining, complete, next and items. Counts refer to records consumed from that snapshot. A returned item is either a complete inline record or a content-addressed reference to one. Thus returned counts entries, not necessarily inline records. The consumer must resolve references before declaring full reconstruction.

The implementation serializes application JSON compactly with ensure_ascii=False, measures its UTF-8 bytes, and adds whole records while the complete envelope fits. It never crops a serialized result. When a single record cannot fit, the page carries its ID, SHA-256 resource reference and byte length. The local resource store supplies the full record, and the client verifies its digest before decoding. If even an empty envelope or a required reference cannot fit, the implementation raises an explicit error.

The snapshot ID is a 16-hex-character prefix of the serialized source's SHA-256 digest. Cursors are 24-character digest-derived keys held in a server-side offset map; the client treats them as opaque. These shortened IDs are deterministic demonstration identifiers, not authentication tokens or collision-resistant security guarantees for an exposed service. Unknown cursors are rejected. Production code should use a suitably strong identifier policy and protected, expiring server-side cursor state.

The budget excludes the outer MCP/JSON-RPC envelope, duplicated text/structured representations, transport framing and model tokens. An integration must reserve those costs independently and remeasure the final delivered representation. A 512-byte application payload is not a 512-token message. Resources are read outside the bounded page channel; their bytes are reported rather than hidden in the claim of budget compliance.

The supplied CoverageStore is an in-memory Python reference adapter. It is not a deployed MCP server, a persistent storage service, a streaming parser or an authentication system. The prototype assumes the source object remains immutable and the process retains its cursor/resource maps. It does not enforce immutability against arbitrary in-process mutation.

4. Experimental method

The experiment enumerates the Cartesian product of four record counts (1, 8, 32, 128), four text widths (32, 128, 512, 2,048 repeated code points per record), four alphabets (ASCII a, accented é, CJK U+4E2D and emoji U+1F600), four budgets (512, 1,024, 2,048, 4,096 bytes), and two top-level field orders. This yields 512 configurations, including 256 paired field-order variants. JSON records have deterministic integer IDs and repeated text; the only fixed non-record evidence is the synthetic proof object.

Four policies receive the same source and nominal budget. Byte prefix slices compact UTF-8 bytes. Character prefix slices the compact Unicode string to the same numeric limit, then encodes it. Record prefix retains proof and as many complete records as fit, with no count, omission flag or retrieval path. Coverage applies the contract above and an exhaustive scripted client. The record-prefix baseline is intentionally a minimal valid-prefix policy, not a claim about all production pagination implementations.

Every first response is decoded with Python's JSON parser. We record valid JSON, first-response byte compliance, proof preservation, explicit coverage fields and first-page entry count. Complete recovery requires exact list equality with the source records and exact proof equality. Prefix policies have no continuation mechanism; coverage is followed to termination with cycle detection, monotonic offsets, count checks, stable snapshot checks and resource-hash verification. Resource contents and their bytes are included in recovery accounting, not in the bounded-page size.

The script writes one row per policy/configuration to results.csv and aggregate counts to summary.json. The measured run used Python 3.12.14. There are no network calls, model calls, random seeds, learned scorers or patient data. Run python3 benchmark.py followed by python3 checks.py from the supplied directory. The same finite inputs should reproduce the same result counts; Python-version text in summary.json may differ.

5. Results

Counts below have denominator 512 for each policy. Recovery for coverage includes all continuation and resource reads; the other policies use their single delivered response.

Policy	Valid JSON	Within byte budget	Proof preserved	Coverage declared	Exact recovery
Byte prefix	120	512	120	0	120
Character prefix	160	208	160	0	160
Record prefix	512	512	512	0	120
Coverage envelope	512	512	512	512	512

Byte-prefix JSON validity is 23.44%; character-prefix validity is 31.25%. The latter's apparent recovery advantage is confounded by exceeding the byte budget in 304 cases. It is not a superior equal-byte-budget result. Valid byte and record prefixes reconstruct the complete source in 120 cases (23.44%). Record-prefix parsing alone would conceal the fact that 392 configurations omit source records.

The coverage envelope reaches exact reconstruction in the entire finite suite. This is expected from its explicit retrieval mechanism and exhaustive client; it should be read as a tested conformance property, not an empirical model-performance advantage. Mean first-page entries are 1.16, 1.94, 4.28 and 4.41 for byte, character, record and coverage policies respectively. Coverage's entry count includes references and is therefore not directly equivalent to inline evidence density.

Coverage requires a mean of 25.5977 page calls, a maximum of 128, and a mean of 16.5039 external resource reads. Mean external resource content is 61,406.98 bytes per configuration. These large costs arise from the full matrix, including 128 long multibyte records at small page budgets. They show why a system should not force exhaustive retrieval for every task: a client should request the subset required by its decision, or use a separate bulk artifact path. This experiment tests completeness when requested, not the optimal retrieval policy.

Five additional adversarial checks pass: an empty snapshot terminates correctly; an envelope smaller than its required metadata is rejected; a fabricated cursor is rejected; a corrupted resource fails digest validation; and a cursor from a different snapshot is rejected by a fresh store. These checks are executable examples, not a comprehensive security audit or proof against all malformed inputs.

6. Companion engineering case

The motivating application is an orchestration node returning a bounded list and a workflow status. A downstream node must distinguish an incomplete list from an exhausted search and a returned resource handle from evidence already read. This can be integrated into a FastAPI tool endpoint or a LangGraph tool node, but no such deployment is claimed here.

The practical change is to budget the complete serialized object, preserve essential control fields explicitly and expose remaining coverage as first-class state. A completion gate can require remaining=0 for exhaustive tasks, or record the inspected subset for selective tasks. It must also resolve referenced records and check source authority separately; a checksum establishes byte integrity, not the truth of a provider's assertions.

In a production integration, the adapter should sit before the final harness serialization. Measure bytes or tokens again after wrappers and text duplication. Track source items, delivered items, outstanding resources, page calls, retrieved bytes, stale-cursor failures and final-state verification separately. These are proposed integration measures, not measured production benefits.

7. Limitations and threats to validity

Synthetic repeated strings isolate encoding and size effects but do not represent real query distributions, semantic relevance or model attention. The grid deliberately weights every factor combination equally; its aggregate percentages are not estimates of real-world failure rates. Field-order variants count as separate deterministic cases, not independent real-world samples. There are no confidence intervals because the complete specified grid is enumerated.

The coverage client is instructed to retrieve everything and cannot decide to stop early. Language models may ignore cursors, use partial information successfully or fail even with all records. No conclusion about their behavior follows. The implementation's exhaustive pack-and-reserialize approach can be expensive on large inputs; no runtime or memory benchmark was performed. It lacks persistent snapshots, authorization, expiration, access logging, revocation, tenant isolation, concurrency handling and bounded resource streaming. SHA-256 resource integrity checks do not replace these controls.

The fixed proof field is easy to retain and models only one class of control metadata. Other applications may need nested provenance, uncertain totals or partial provider failures, which this contract does not cover. Coverage declarations are trusted producer statements and require independent checks where trust is limited. New transport adapters should be tested with actual schemas, real error distributions and complete harness wrappers before adoption.

8. Conclusion

A byte limit, valid JSON and complete evidence are different properties. The finite experiment demonstrates that preserving whole records repairs syntax without communicating omissions, and that explicit coverage plus retrieval can preserve reconstruction while imposing substantial extra work. The reference implementation offers an inspectable starting point for transport checks in agent systems. Downstream agent accuracy, selective retrieval quality and production operational performance remain future evaluation tasks.

Availability, authorship and disclosures

Source, raw CSV, summaries, adversarial checks and the companion case study are distributed in https://github.com/samihalawa/research-publications/tree/main/bounded-tool-results . Documents are offered under CC BY 4.0; reference code under the MIT license included alongside it. Only synthetic records are used. No participant study, medical experiment or external paid model evaluation was performed. No external funding or competing commercial result is asserted.

This report and implementation were prepared using AI assistance under Sami Halawa Ribas's direction. The assistant generated and executed the reference code, assembled the manuscript and checked deterministic outputs. This disclosure does not represent independent human or peer review. The report is a technical report, not a peer-reviewed publication. The author's role in this run is defining the research-to-publication objective and directing the engineering scope; code generation and manuscript preparation used AI assistance.

References

[1] Sehgal, Y., Panahandeh, M., and Ezzati-Jivan, N. (2026). Context Is a Pipeline, Not a Window: Information Attrition in LLM Agent Harnesses. arXiv:2610.36114v1, 26 October 2026. https://arxiv.org/abs/2610.36114

[2] Model Context Protocol contributors (2025). Tools. Specification revision 2025-06-18. Accessed 1 November 2026. https://modelcontextprotocol.io/specification/2025-06-18/server/tools

[3] Model Context Protocol contributors (2025). Pagination. Specification revision 2025-06-18. Accessed 1 November 2026. https://modelcontextprotocol.io/specification/2025-06-18/server/utilities/pagination